CENTRAL PHILIPPINE UNIVERSITY
College of Computer Studies
Iloilo City

ALOHAR MiniOS
Individual Contribution Report

Subject: System Administration and Maintenance
Subject: IT3131 (SC: 1063)

Submitted by:
Group ALOHAR

Hanna Alexandra D. Caballe – Project Leader
Luis Raphael P. Argel – CLI Developer
Ondrej Stefano Cres F. Carpio – File Management Developer
Aron Adriel T. Siason – Documenter
Reynaldo D. Talabucon II – Process and Memory Developer
Annyleise S. Torrechilla – User/System Developer

Submitted to:
A.M.P. Latter P. Taasan
Instructor

Submitted on:
October 24, 2026

Student Name: Hanna Alexandra D. Caballe
Group: ALOHAR
Assigned Role: Project Leader

What did I contribute to the project?

- As the group leader, I am responsible for delegating tasks, coordinating my group members, and ensuring that the deadline is met. Additionally, I took the initiative to do the **README.md** and **User Manual** to lessen the tasks of our assigned documenter, as well as editing our demo video.

What features did I personally develop?

- N/A. My primary contribution was more on project and member management, documentation, and video editing rather than feature development.

What problems did I encounter?

- There were no severe problems or anything that was close to being called a “problem” but coordinating our schedules and ensuring that all group members completed their tasks on time were something to focus on as a group leader.

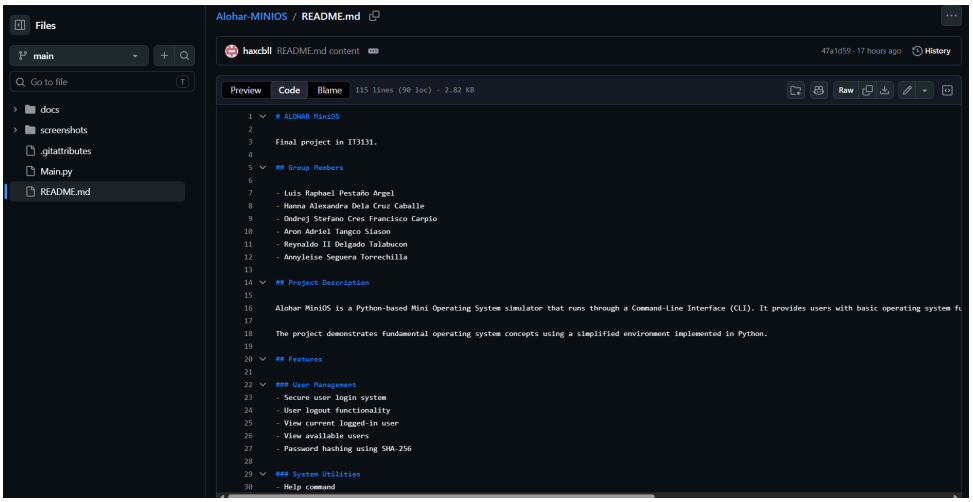
How did I solve those problems?

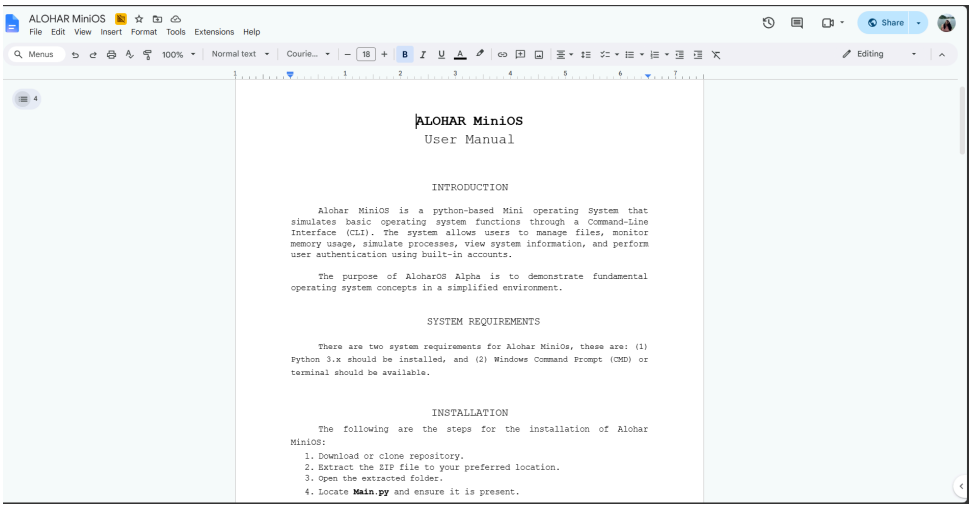
- I communicated with my group members in our group chat, monitored their progress, and sent them reminders from time to time.

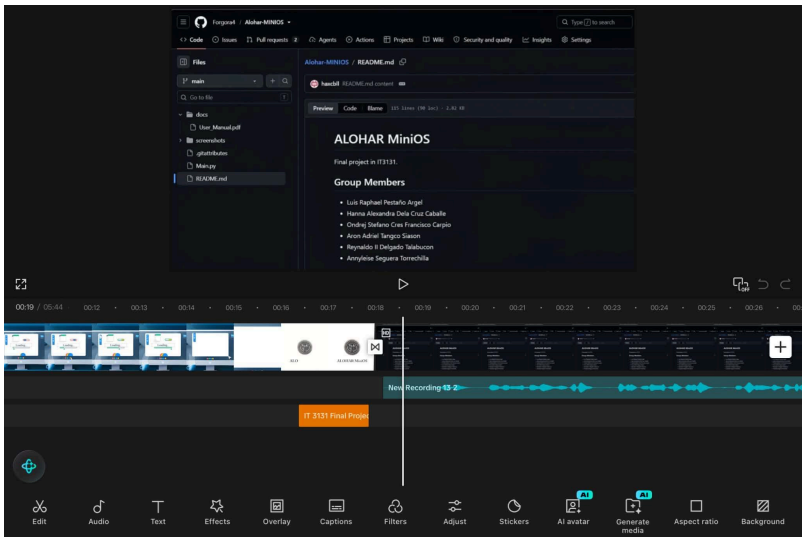
What did I learn from this project?

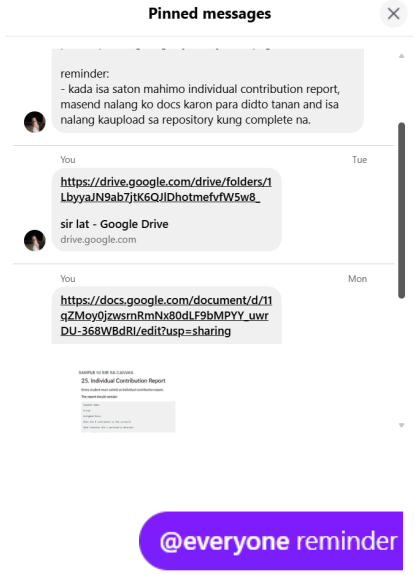
- I learned the importance of task delegation, teamwork, and clear communication to successfully complete a software project. All my members were reliable and I learned that effective leadership involves trusting team members.

Evidence of my contribution (Next Page)


Files
main
Go to file
docs
screenshots
.gitattributes
Main.py
README.md
Alohar-MINIOS / README.md
haxcbll README.md content
47a1d59 · 17 hours ago
History
Preview
Code
Blame
115 lines (90 loc) · 2.82 KB
Raw
ALOHAR MiniOS
Final project in IT3131.
Group Members
- Luis Raphael Pestaño Argel
- Hanna Alexandra Dela Cruz Caballe
- Ondrej Stefano Cres Francisco Carpio
- Aron Adriel Tangco Siason
- Reynaldo II Delgado Talabucon
- Annyleise Seguera Torrechilla
Project Description
Alohar MiniOS is a Python-based Mini Operating System simulator that runs through a Command-Line Interface (CLI). It provides users with basic operating system fu
The project demonstrates fundamental operating system concepts using a simplified environment implemented in Python.
Features
User Management
- Secure user login system
- User logout functionality
- View current logged-in user
- View available users
- Password hashing using SHA-256
System Utilities
- Help command


ALOHAR MiniOS
File Edit View Insert Format Tools Extensions Help
Share
Editing
ALOHAR MiniOS
User Manual
INTRODUCTION
Alohar MiniOS is a python-based Mini operating System that simulates basic operating system functions through a Command-Line Interface (CLI). The system allows users to manage files, monitor memory usage, simulate processes, view system information, and perform user authentication using built-in accounts.
The purpose of AloharOS Alpha is to demonstrate fundamental operating system concepts in a simplified environment.
SYSTEM REQUIREMENTS
There are two system requirements for Alohar MiniOs, these are: (1) Python 3.x should be installed, and (2) Windows Command Prompt (CMD) or terminal should be available.
INSTALLATION
The following are the steps for the installation of Alohar MiniOS:
1. Download or clone repository.
2. Extract the ZIP file to your preferred location.
3. Open the extracted folder.
4. Locate Main.py and ensure it is present.


ALOHAR MiniOS
Final project in IT3131.
Group Members
Luis Raphael Pestaño Argel
Hanna Alexandra Dela Cruz Caballe
Ondrej Stefano Cres Francisco Carpio
Aron Adriel Tangco Siason
Reynaldo II Delgado Talabucon
Annyleise Seguera Torrechilla
New Recording 13 2
IT 3131 Final Proje
Edit
Audio
Text
Effects
Overlay
Captions
Filters
Adjust
Stickers
AI avatar
Generate media
Aspect ratio
Background


Pinned messages
reminder:
- kada isa saton mahimo individual contribution report, masend nalang ko docs karon para didto tanan and isa nalang kaupload sa repository kung complete na.
You
Tue
https://drive.google.com/drive/folders/1LbyyaJN9ab7jtK6QJlDhotmefvfW5w8_
sir lat - Google Drive
drive.google.com
You
Mon
https://docs.google.com/document/d/11qZMoy0jzwsrnRmNx80dLF9bMPYY_uwrDU-368WBdRI/edit?usp=sharing
@everyone reminder

Student Name: Luis Raphael P. Argel
Group: ALOHAR
Assigned Role: CLI Developer

What did I contribute to the project?

- I contributed the main structure and foundation of the CLI for our project, AloharOS. My role was to create the base program that lets the operating system run continuously and accept commands from the user. I am also the person responsible for the main repository of our code in github.

What features did I personally develop?

- I set up the main Python class for AloharOS, which holds all the basic settings like the system boot time. I also created the interactive command prompt where users type input, the dynamic list that holds all our system commands, and the command that lets users safely close the program.

What problems did I encounter?

- As a Jr. developer, my main challenge was figuring out how to keep the program running in a loop without it crashing when someone typed a wrong command. I also needed to find an easy way for my teammates to add their own commands without breaking my main setup.

How did I solve those problems?

- I used a “While” loop with a simple true/false variable so the program stays open until the user types "exit". I also created a command list using a Python dictionary that can allow everyone to connect their own functions directly to the system. I also cleaned up user inputs using standard Python functions so the system wouldn't crash on empty or unexpected typing.

What did I learn from this project?

- I learned how command-line programs stay open and process user input in real time. Also I learned how to organize Python code using classes so that my team can work together and build different parts of a project onto one shared structure.

Evidence of my contribution (Next Page)

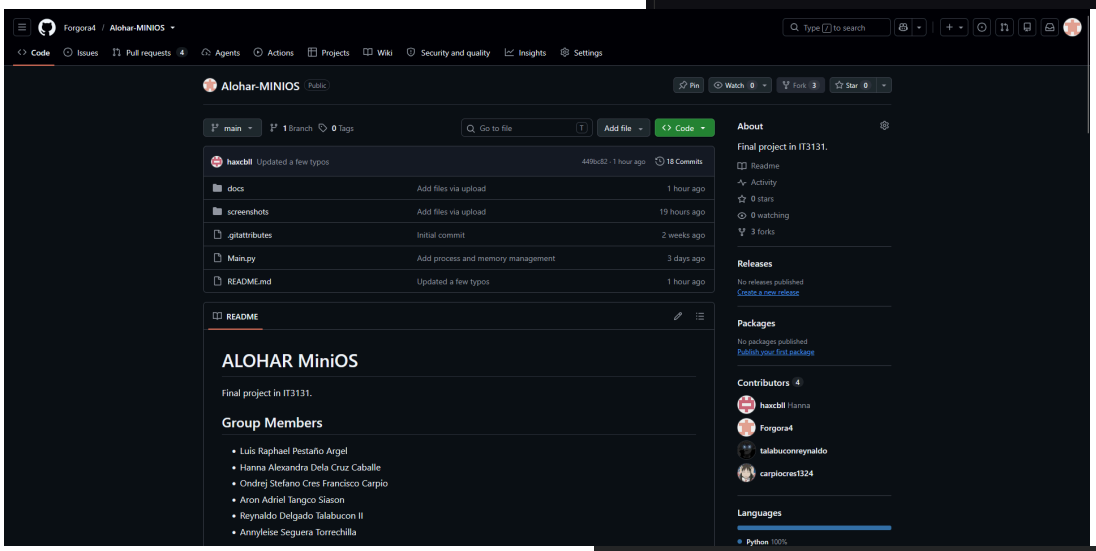
```python
    #Diri kamo add new commands, ninyo try lang follow format ko like def
    # cd(self, args):
    # Command to change directory

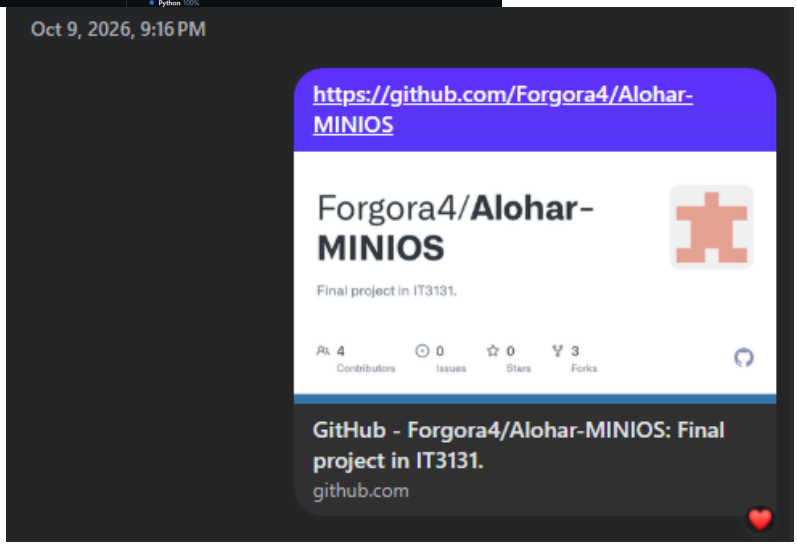


if __name__ == "__main__":
    app = alohar()
    app.run()
```

```python
#Luis: Main CLI framework
class alohar:
    def __init__(self, hostname = "aloharos", username="root"):
        self.hostname = hostname
        self.username = username
        self.is_authenticated = False
        # Demo accounts: admin/admin and student/student.
        self.users = {
            "admin": hashlib.sha256("admin".encode()).hexdigest(),
            "student": hashlib.sha256("student".encode()).hexdigest()
        }
        self.boot_time = time.time()
        self.is_running = True
        self.processes = {}
        self.next_pid = 1
        self.total_memory = 4096
        self.used_memory = 0

        self.cmd = {
            "help": self.cmdhelp,
            "clear": self.cmdclear,
            "uptime": self.uptime,
            "time": self.show_time,
            "whoami": self.whoami,
            "users": self.list_users,
            "login": self.login,
            "logout": self.logout,
            "systeminfo": self.systeminfo,
            "exit": self.exit,
            "process": self.process,
            "run": self.run_process,
            "kill": self.kill,
            "memory": self.memory,
            "allocate": self.allocate,
            "free": self.free,
            "pwd": self.pwd,
            "ls": self.ls,
            "cd": self.cd,
            "cat": self.cat
            # diri ka add sang new commands ninyo like "cd": self.cd
        }
```





Student Name: Ondrej Stefano Cres F. Carpio
Group: ALOHAR
Assigned Role: File Management Developer

What did I contribute to the project?

- I contribute to the development of the File Management Developer of AloharOS. My key responsibility is to create file management for the user so they can navigate, add, delete, read to files and folders.

What features did I personally develop?

- I developed the pwd, is, cd this command is for navigating through files viewing by using ls on list. I added to create write and read (as a cat) and delete for managing the file management along with a pair to create and delete of mkdir and rmdir i handle the error in case of missing files.

What problems did I encounter?

- I have a challenge to merge my code with my teammates code since we split work in the same python file and manage the code to not run into a problem by handling the invalid file on the OS.

How did I solve those problems?

- by learning the latest version of the repository, adding the file management function and commands correcting the code so it can't run into error by creating a file then testing it so it can run smoothly creating files.

What did I learn from this project?

- I learn how to collaborate with my teammates on how to build a custom OS and build a command line interface. and improving my debugging skill, and learning more about GitHub.

Evidence of my contribution (Next Page)

```python
def pwd(self, args):
    print(os.getcwd())


def ls(self, args):
    directory = " ".join(args) if args else "."
    try:
        items = os.listdir(directory)
        for item in sorted(items):
            print(f"  {item}")
    except FileNotFoundError:
        print(f"Directory not found: {directory}")
    except NotADirectoryError:
        print(f"Not a directory: {directory}")
    except PermissionError:
        print(f"Permission denied: {directory}")
    except OSError as e:
        print(f"Error listing directory: {e}")


def cd(self, args):
    if not args:
        print("Usage: cd <directory>")
        return
    directory = " ".join(args)
    try:
        os.chdir(directory)
        print(f"Moved to: {os.getcwd()}")
    except FileNotFoundError:
        print(f"Directory not found: {directory}")
    except NotADirectoryError:
        print(f"Not a directory: {directory}")
    except PermissionError:
        print(f"Permission denied: {directory}")
```

```python
"pwd": self.pwd,
"ls": self.ls,
"cd": self.cd,
"cat": self.cat,
"read": self.cat,
"create": self.create_file,
"write": self.write_file,
"delete": self.delete_file,
"mkdir": self.make_directory,
"rmdir": self.remove_directory
```

```python
def cat(self, args):
    if not args:
        print("Usage: cat <filename>")
        return
    filename = " ".join(args)
    try:
        with open(filename, "r", encoding="utf-8") as f:
            print(f.read())
    except FileNotFoundError:
        print(f"File not found: {filename}")
    except IsADirectoryError:
        print(f"'{filename}' is a directory, not a file.")
    except PermissionError:
        print(f"Permission denied: {filename}")
    except OSError as e:
        print(f"Error reading file: {e}")


def create_file(self, args):
    if not args:
        print("Usage: create <filename>")
        return

    filename = " ".join(args)
    try:
        with open(filename, "x", encoding="utf-8"):
            pass
        print(f"File created: {filename}")
    except FileExistsError:
        print(f"File already exists: {filename}")
    except PermissionError:
        print(f"Permission denied: {filename}")
    except OSError as e:
        print(f"Error creating file: {e}")


def write_file(self, args):
    if len(args) < 2:
        print('Usage: write <filename> "<text>"')
        return
```

Student Name: Aron Adriel T. Siason
Group: ALOHAR
Assigned Role: Documenter

What did I contribute to the project?

- Compiled, organized, and wrote the comprehensive technical documentation outlining how the Alohar - MINIOS simulated operating system works from the ground up.
- Documented all twelve core sections of the project, including the system architecture, program structure, core subsystems, testing procedures, and solutions to development challenges.

What features did I personally develop?

- As the Documenter, my primary deliverables were the technical documentation text, system workflow documentation, and structural breakdowns of the codebase rather than writing functional runtime features.

What problems did I encounter?

- Synthesizing and structuring complex technical details across multiple programming subsystems (such as command routing dictionaries, memory tracking, and process states) into a clear, unified document.

How did I solve those problems?

- Collaborated closely with the project leader and developers to review code implementation details, structure each of the 12 documentation sections logically, and verify technical accuracy against the final program build.

What did I learn from this project?

- Gained a comprehensive, end-to-end understanding of how operating system command shells process inputs, manage simulated RAM, handle authentication, and interact with file systems.

Evidence of my contribution: (separate page)

Alohar - MINIOS Technical Documentation

Course: IT3131
Institution: Central Philippine University
Project Title: Alohar – MINIOS: Simulated Python Operating System
Group Name: Alohar – MINIOS Group

GROUP MEMBERS

- Caballe – Project Leader
- Argel– CLI Developer
- Carpio – File Management Developer
- Talubucon– Process and Memory Developer
- Torechilla – User/System Developer
- Siason – Documenter

1. INTRODUCTION

Alohar – MINIOS is a lightweight, command-line simulated operating system developed entirely in Python. Created as a final project for IT3131, the program demonstrates how an operating system manages core tasks behind the scenes without requiring actual hardware-level changes.

Through a terminal interface, users can securely log in, navigate the host machine's directories, start and terminate simulated background tasks, and allocate virtual system memory.

2. PROJECT OBJECTIVES

The main objectives of developing AloharOS are:

- To understand the core concepts of an operating system by building a working simulation using object-oriented Python.
- To implement a functional command-line interface (CLI) that continuously reads user input, sanitizes it, and routes it to the appropriate functions.
- To simulate RAM allocation and task management logic safely within an isolated environment.
- To interact with the actual host file system using standard Python OS libraries while preventing program crashes through proper error handling.

3. SYSTEM ARCHITECTURE

Alohar - MiniOS follows a monolithic architecture built around a single primary Python class named alohar. The central command shell serves as the router between the user and the core system functions.

When a user enters a command, the Command Shell processes the input, references a command dictionary, and triggers the appropriate subsystem.



The system consists of the following components:

- **Command Shell:** Processes user commands and routes them to the appropriate methods.
- **File System:** Handles directory navigation and file viewing through the pwd, ls, cd, and cat commands.
- **Process Management:** Simulates running tasks through the run, kill, and process commands.
- **Memory Management:** Simulates memory allocation and release through the allocate, free, and memory commands.
- **User and System Information:** Manages authentication, user sessions, and system information through the login, logout, whoami, systeminfo, and uptime commands.

4. PROGRAM STRUCTURE

To maintain a unified and easily executable program, the entire system is structured inside a single Python script named Main.py.

Main.py

The Main.py file serves as the program's entry point. It initializes system variables, including the boot time, memory limits, and built-in user accounts, and starts the command-line interface loop.

The run() Loop

The run() method contains a while self.is_running: loop that acts as the heartbeat of the operating system. It continuously waits for user input, separates the command from its arguments, and routes the request to the appropriate function.

Command Dictionary (self.cmd)

Instead of relying on numerous if-else statements, Alohar - MINIOS uses a dictionary to connect command strings, such as help, directly to their corresponding Python methods.

This approach makes command routing more organized, readable, and easier to maintain.

5. FILE MANAGEMENT

File management connects the simulated operating system to the actual host computer's file system using Python's standard os library.

Directory Navigation

- **pwd:** Uses os.getcwd() to retrieve and display the current working directory.
- **cd:** Uses os.chdir() to change the current working directory.
- **ls:** Uses os.listdir('.') to retrieve and display the files and folders in the current directory.
- **cat:** Uses Python's open(file, 'r') function to read and display the contents of text files in the CLI.

Error Handling and Safety

File management commands are wrapped in try-except blocks to handle common errors, including FileNotFoundError, NotADirectoryError, and PermissionError.

These exception handlers prevent the program from terminating unexpectedly when a user enters an invalid path, attempts to access an inaccessible file, or provides an incorrect directory.

6. PROCESS MANAGEMENT

Process management simulates how an operating system's Task Manager tracks running applications.

Process Storage

Alohar - MINIOS uses the self.processes dictionary to store simulated background tasks. Each process is assigned a unique Process ID (PID), which serves as the dictionary key.

Starting Processes

The run <name> command starts a simulated process. The system assigns the current value of self.next_pid, records the process with a RUNNING status, and increments the PID counter to ensure that newly created processes receive different identifiers.

Terminating Processes

The kill <PID> command removes the specified process from the dictionary, simulating the termination of an application. The PID is not reused during the same session.

Displaying Processes

The process command iterates through the process dictionary and displays a formatted table of all active simulated tasks.

7. MEMORY MANAGEMENT

The memory management module simulates system RAM using basic arithmetic and integer variables.

Memory Capacity

The system starts with a predefined memory capacity of 4096 MB. The self.total_memory variable stores the total capacity, while self.used_memory tracks the amount of memory currently allocated. Initially, the used memory is set to 0 MB.

Memory Allocation

The allocate <MB> command converts the user's input into an integer and checks whether the requested amount exceeds the available memory.

If sufficient memory is available and the request is valid, the system adds the requested amount to self.used_memory. Otherwise, the allocation is rejected.

Memory Release

The free <MB> command releases previously allocated memory back into the available memory pool.

The system checks whether the requested release exceeds the currently used memory. This prevents the used-memory counter from becoming negative.

8. USER MANAGEMENT

User management handles authentication and session tracking during system operation.

Authentication

Alohar – MINIOS stores demonstration accounts, including admin and student, in a dictionary. Passwords are hashed using Python's hashlib.sha256() rather than being stored as plain text.

The system uses hmac.compare_digest() to compare password hashes during authentication.

Session Locking

When the program starts, the run() loop calls the self.login() method. The user must provide a valid username and password before accessing the operating system's commands.

The getpass library hides password input in the terminal, preventing the password characters from being displayed while the user types.

User Tracking

After successful authentication, the username is stored in self.username. The system can retrieve and display the active username through the whoami and systeminfo commands.

9. ERROR HANDLING

AloharOS incorporates error-handling mechanisms to maintain stability and prevent common user-input mistakes from interrupting program execution.

Input Sanitization

User input is processed using .strip() to remove unnecessary leading and trailing spaces. The .lower() method is used to make command recognition case-insensitive.

Empty Input Handling

If the user presses Enter without entering a command, the program skips the current loop iteration. This prevents errors that could occur when attempting to process an empty input.

Unknown Commands

When a command does not exist in the self.cmd dictionary, the system displays a message indicating that the command is unknown. The CLI loop then continues running, allowing the user to enter another command.

Invalid Arguments

Commands that require additional arguments check whether the necessary parameters have been provided. For example, the cd command requires a directory path.

These checks prevent missing arguments from causing unexpected errors or interrupting program execution.

10. TESTING

The development team tested Alohar – MINIOS using different input scenarios in the terminal to verify its functionality, error handling, and stability.

Authentication Testing

The team attempted to log in using incorrect passwords to verify that unauthorized access was rejected. Password input was also checked to ensure that characters were hidden while typing.

File System Testing

The team tested directory navigation and file reading by moving between directories and attempting to access nonexistent folders or read directories as files.

These tests verified that the appropriate exception handlers responded to invalid file system operations.

Memory and Process Boundary Testing

The team tested process termination by attempting to kill nonexistent PIDs. Memory management was tested by entering non-numeric values into the allocate command and attempting to allocate more than the available 4096 MB capacity.

These scenarios verified that invalid inputs and requests exceeding system limits were handled appropriately.

11. PROBLEMS ENCOUNTERED AND SOLUTIONS

During development, the team encountered several logic and structural challenges. These were addressed through debugging and improvements to the program's input handling and command routing.

Infinite Loops and Errors Caused by Missing Arguments

In early versions of the program, entering a command without the required arguments, such as typing cd without specifying a directory, could cause errors that interrupted program execution.

This issue was addressed by adding argument checks, such as if not args:, at the beginning of functions that require parameters. These checks ensure that the program handles incomplete commands safely.

Dictionary-Based Command Routing

Implementing the self.cmd dictionary required the program to pass the arguments list consistently to each command function.

Even commands that did not require additional arguments, such as clear or pwd, needed compatible function definitions to avoid parameter mismatches.

The team resolved this structural issue by ensuring that command methods followed a consistent parameter pattern, making the command-routing mechanism more reliable and maintainable.

12. CONCLUSION

Developing Alohar - MINIOS provided the team with practical experience in understanding how operating systems use command shells to process user input, manage simulated memory, authenticate users, and interact with file systems.

The project also strengthened the team's understanding of object-oriented programming in Python, exception handling, dictionaries, and collaborative software development.

Through the implementation of its command-line interface and core subsystems, Alohar – MINIOS demonstrates fundamental operating system concepts in a simplified and accessible environment. Overall, the project served as a valuable learning experience in applying programming concepts to the simulation of an operating system.

Student Name: Reynaldo D. Talabucon II
Group: ALOHAR
Assigned Role: Process and Memory Developer

What did I contribute to the project?

- I contributed to the development of the Process and Memory Management features of AloharOS. My responsibility was to create Python functions that simulate how an operating system handles processes and memory. I also helped integrate and test these functions to make sure they worked properly with the existing system.

What features did I personally develop?

- I developed six commands: **run, process**, and **kill** for Process Management, and **memory**, **allocate**, and **free** for Memory Management. These commands allow users to start, view, and terminate simulated processes, as well as monitor, allocate, and release simulated memory. I also added error handling for invalid process IDs and memory operations.

What problems did I encounter?

- One of the problems I encountered was integrating my code with my groupmates' existing code. I experienced GitHub merge conflicts because we were working on the same Python file. I also encountered indentation and spacing issues while adding my functions.

How did I solve those problems?

- I used the latest version of our repository, added only my assigned functions, corrected the formatting issues, and tested the commands before uploading my changes.

What did I learn from this project?

- I learned how process and memory management work through simulation. I also improved my Python programming, debugging, and GitHub collaboration skills.

Evidence of my contribution (Next Page)

Evidence of my contribution:




```
[22:15:12] AloharOS# run calculator
Process started.
PID: 1
Name: calculator
Status: RUNNING
```

```
[22:16:39] AloharOS# memory

MEMORY STATUS
------------------------------
Total Memory: 4096 MB
Used Memory:  0 MB
Free Memory:  4096 MB
```

```
[22:15:41] AloharOS# process

PROCESS LIST
--------------------------------------
PID     NAME                STATUS
--------------------------------------
1       calculator          RUNNING
```

```
[22:16:24] AloharOS# process
No running processes.
```

```
[22:16:05] AloharOS# kill 1
Process 1 terminated.
```

```
[22:20:24] AloharOS# allocate 500
500 MB allocated.
```

Student Name: Annyleise S. Torrechilla
Group: ALOHAR
Assigned Role: User/ System Developer

What did I contribute to the project?

- I contributed the **User Management** and **System Information** parts of AloharOS. I made sure users must log in before using the system, and I added a command that displays the system's simulated hardware and OS details.

What features did I personally develop?

- I developed the **login**, **logout**, **whoami**, and **users** commands, along with the demo accounts (admin and student) whose passwords are stored as SHA-256 hashes. I also developed the **systeminfo** command, which shows the OS name, version, architecture, CPU, memory, system time, and current user.

What problems did I encounter?

- At first, the password showed on the screen while I was typing it in the login. I also had a small problem making the boxes line up properly in the login and system information screens.

How did I solve those problems?

- I used **getpass** so the password stays hidden when typing. For the boxes, I tested the spacing several times and used centered text formatting until it looked neat.

What did I learn from this project?

- I learned how a command-line interface reads input and runs commands, how basic authentication and password hashing work, and how to manage login sessions. I also learned how to write code that fits into a team project.

Evidence of my contribution (Next Page)

Evidence of my contribution

User Management

```
class alohar:
    def __init__(self, hostname = "aloharos", username="root"):
        self.hostname = hostname
        self.username = username
        self.is_authenticated = False
        # LIS Demo accounts: admin/admin and student/student.
        self.users = {
            "admin": hashlib.sha256("admin".encode()).hexdigest(),
            "student": hashlib.sha256("student".encode()).hexdigest()
        }
        self.boot_time = time.time()
        self.is_running = True
```

```
    def run(self):
        # LIS: User Management - require an account login before using AloharOS.
        self.login()
        print("aloharOS CLI")
        print("help for commands ")

        while self.is_running:
            if not self.is_authenticated:
                self.login()
```

```
    # LIS: User Management - authenticate against the built-in demo accounts.
    def login(self, args=None):
        if self.is_authenticated:
            print(f"Already logged in as {self.username}. Log out first.")
            return

        while not self.is_authenticated:
            print("\n" + "+" + "-" * 38 + "+")
            print(f"|{' ALOHAROS LOGIN ':^38}|")
            print("+" + "-" * 38 + "+")
            username = input("\nUsername: ").strip()
            password = getpass.getpass("Password (input hidden): ")
            password_hash = hashlib.sha256(password.encode()).hexdigest()

            if username in self.users and hmac.compare_digest(
                self.users[username], password_hash
            ):
                self.username = username
                self.is_authenticated = True
                print("\nLogin successful.")
                print(f"\nWelcome, {self.username}!")
                return

            print("Invalid username or password. Please try again.\n")
```

```
    def logout(self, args):
        if not self.is_authenticated:
            print("No user is currently logged in.")
            return

        print(f"Goodbye, {self.username}.")
        self.username = None
        self.is_authenticated = False

    def whoami(self, args):
        if not self.is_authenticated:
            print("No user is currently logged in.")
            return
        print(f"Current User: {self.username}")

    def list_users(self, args):
        print("Available users:")
        for username in sorted(self.users):
            print(f"- {username}")
```

System Information

```
    # LIS: System Information - simulated AloharOS hardware details.
    def systeminfo(self, args):
        architecture = f"{struct.calcsize('P') * 8}-bit"
        current_user = self.username if self.is_authenticated else "None"
        border = "+" + "-" * 42 + "+"
        print(border)
        print(f"|{' ALOHAROS SYSTEM INFORMATION ':^42}|")
        print(border)
        print("\nOS Name: AloharOS Alpha")
        print("Version: 1.0.0")
        print(f"Architecture: {architecture}")
        print("CPU: AloharOS Virtual CPU (4 cores)")
        print("Memory: 4096 MB")
        print(f"System Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
        print(f"User: {current_user}")
        print(f"\n{border}")
```

Others

